

Muhammad Saad Khan (251309919)

1.

$$F_n = F_{n-1} + F_{n-2}, n > 1; F_1 = 1; F_0 = 0.$$

Let $A = (1 + \sqrt{5})/2$. Using the equation $A^2 = A + 1$, prove that

$$A^{n-2} \leq F_n \leq A^n, \text{ for } n \geq 1.$$

proof:

base cases:

$$n = 1: F_1 = 1 \text{ and } A^{-1} \leq 1 \leq A$$

$$n = 2: F_2 = 1 \text{ and } A^0 = 1 \leq 1 \leq A^2$$

$$\text{assume } A^{n-2} \leq F_n \leq A^n \text{ and } A^{n-3} \leq F_{n-1} \leq A^{n-1}$$

inductive step:

$$F_{n+1} = F_n + F_{n-1}$$

upper bound:

$$F_{n+1} \leq A^n + A^{n-1}$$

$$= A^{n-1}(A + 1)$$

$$= A^{n-1}A^2$$

$$= A^{n+1}$$

lower bound:

$$F_{n+1} \geq A^{n-2} + A^{n-3}$$

$$= A^{(n-3)}(A + 1)$$

$$= A^{(n-3)}A^2$$

$$= A^{(n-1)}$$

$A^{(n-1)} \leq F_{n+1} \leq A^{(n+1)}$, completing the induction

b)

$$A^{(n-2)} \leq F_n \leq A^n,$$

$$(n-2)\log_2(A) \leq \log_2(F_n) \leq n\log_2(A)$$

$$\log_2(A) \approx 0.694:$$

F50:

$$33.3 \leq \log_2(F_{50}) \leq 34.7$$

F50 needs about 50 bits

F500:

$$345.6 \leq \log_2(F_{500}) \leq 347.0$$

And F500 needs 348

2)

$$\text{a) worst case} = \Theta(k^2)$$

$$\text{total time} = (n/k) \cdot \Theta(k^2) = \Theta(nk)$$

b) n/k sorted sublists each of length k

each merge accounts for all n elements

$$\text{number of levels} = \log(n/k)$$

total merge time = $\Theta(n \log(n/k))$

c) total run time = $\Theta(nk + n \log(n/k))$

standard merge sort time = $\Theta(n \log n)$

$\Theta(n \log n) = nk + n \log(n/k)$

$\Theta(\log n) = k + \log(n/k)$

Largest k which is allowed is $k = \Theta(\log n)$

d) since insertion sort is fast for small arrays k should be chosen as a small value to improve on this characteristic.

3)

Case	O	o	Ω	ω	Θ
A)	yes	yes	no	no	no
b)	yes	yes	no	no	no
c)	no	no	no	no	no
D)	no	no	yes	yes	no
e)	yes	no	yes	no	yes
f)	yes	no	yes	no	yes

4)

$T(n) \leq T(n/2) + c$

$\leq T(n/4) + c + c$

$\leq T(n/8) + c + c + c$

Stops at $(n/2^k) = 2$

$$n = 2^{(k+1)}$$

$$k = \log_2 n - 1$$

$$\text{depth} = \Theta(\log n) = T(n)$$

proof:

$$T(n) = O(\log n)$$

base case:

$$n = 2$$

$$T(2) \leq c$$

Choose $a \geq c/\log 2$

$$T(2) \leq a \log 2$$

Inductive step

$$T(n) \leq T(n/2) + c$$

$$\leq a \log (n/2) + c$$

$$= a \log (n-1) + c$$

$$= a \log n - a + c$$

$$T(n) \leq a \log n$$

5)

c)

```
1 CXX = g++
2 CXXFLAGS = -O2 -Wall
3
4 all: asn1_a asn1_b
5
6 asn1_a: fib_naive.o
7      $(CXX) $(CXXFLAGS) -o asn1_a fib_naive.o
8
9 asn1_b: fib_fast.o
10     $(CXX) $(CXXFLAGS) -o asn1_b fib_fast.o
```

```
nkha33@compute:~/courses/cs3340/asn1$ whoami
nkha33
nkha33@compute:~/courses/cs3340/asn1$ date
Thu Jan 29 05:53:53 PM EST 2020
nkha33@compute:~/courses/cs3340/asn1$ pwd
/home/nkha33/courses/cs3340/asn1
nkha33@compute:~/courses/cs3340/asn1$ time ./asn1_a
F(0) = 0
F(5) = 5
F(10) = 55
F(15) = 610
F(20) = 6765
F(25) = 75805
F(30) = 832040
F(35) = 9227465
F(40) = 102334155
F(45) = 1149832170
F(50) = 12586269025

real    0m12.483s
user    0m12.467s
sys     0m0.025s
nkha33@compute:~/courses/cs3340/asn1$
```

```
1 CXX = g++
2 CXXFLAGS = -O2 -Wall
3
4 all: asn1_a asn1_b
5
6 asn1_a: fib_naive.o
7      $(CXX) $(CXXFLAGS) -o asn1_a fib_naive.o
8
9 asn1_b: fib_fast.o
10     $(CXX) $(CXXFLAGS) -o asn1_b fib_fast.o
```

```
nkha33@compute:~/courses/cs3340/asn1$ whoami
nkha33
nkha33@compute:~/courses/cs3340/asn1$ date
Thu Jan 29 05:53:07 PM EST 2020
nkha33@compute:~/courses/cs3340/asn1$ pwd
/home/nkha33/courses/cs3340/asn1
nkha33@compute:~/courses/cs3340/asn1$ time ./asn1_b
F(0) = 0
F(20) = 6765
F(40) = 102334155
F(60) = 1548008755020
F(80) = 23416728348467685
F(100) = 36422684427261315875
F(120) = 535853925498966640871040
F(140) = 8195590096023584197200408605
F(160) = 12261325953941829308017470209595
F(180) = 18547707809471980212190138521399707700
F(200) = 28057172992510140037611932413038077189525
F(220) = 424520811530999138679690402189754846230915
F(240) = 6420261486372304121098177423871111802307548623680
F(260) = 971183874599339129547649988289594072811688739584170445
F(280) = 14699394908621181489442072451549121105480936813821970707835
F(300) = 222122446296294455207768936180980726656693380640976400079540
F(320) = 336170714981814467266618721945410482790813867716465834363558711365
F(340) = 508525481336662383400096853894215027094861596208384766708711212163875
F(360) = 7602464272610947650807979422397123045240656897999924447028113150135520
F(380) = 116363906534184169880824991550458115903867894486858448770901160580570880172285
F(400) = 1766230886458139664682269453924112507783843833048921518867259289657534504416819675
F(420) = 26527302640877352173655282218075507488962348728013444188181904845563380781650451440
F(440) = 4027881738283407838585813276891176624248463196560667664545975728426578067220435913205
F(460) = 60929766364353889279195197999021728669389417532255086444641219473596279869815908465388209608595
F(480) = 92168057176568747120845496272620415567368565986794771113980581316448136748508616499826192287280
F(500) = 1394212245556977081397243829740720395807026587673072641889524832571102286329692155765867622521294125

real    0m0.001s
user    0m0.000s
sys     0m0.002s
nkha33@compute:~/courses/cs3340/asn1$
```

The naïve approach has an exponential time complexity which leads to the algorithm becoming really slow as n increases due to redundant computations. While the optimized version has a linear time complexity which makes it efficient to compute high Fibonacci numbers.

d) 5a cannot compute f50 using 4 byte int since f50 would result in an overflow. 5b is able to compute f500 so precisely since it uses a custom big integer and stores all digits explicitly

5a algorithm:

```
long long fib(int n) {  
    if (n == 0) return 0;  
    if (n == 1) return 1;  
    return fib(n - 1) + fib(n - 2);  
}
```

```
int main() {  
    for (int i = 0; i <= 10; i++) {  
        int n = i * 5;  
        cout << "F(" << n << ") = " << fib(n) << endl;  
    }  
    return 0;  
}
```

Time complexity = $O(n^2)$

5b algorithm:

```
struct BigInt {  
    vector<int> d;  
  
    BigInt(long long x = 0) {  
        if (x == 0) d.push_back(0);  
    }  
};
```

```

while (x > 0) {
    d.push_back(x % 10);
    x /= 10;
}
}

```

```

BigInt operator+(const BigInt& other) const {
    BigInt res;
    res.d.clear();
    int carry = 0;
    size_t n = max(d.size(), other.d.size());
    for (size_t i = 0; i < n || carry; i++) {
        int sum = carry;
        if (i < d.size()) sum += d[i];
        if (i < other.d.size()) sum += other.d[i];
        res.d.push_back(sum % 10);
        carry = sum / 10;
    }
    return res;
}

```

```

void print() const {
    for (int i = d.size() - 1; i >= 0; i--)
        cout << d[i];
}
};

```

```

pair<BigInt, BigInt> fib_fast(int n) {
    if (n == 0) return {BigInt(0), BigInt(0)};
    if (n == 1) return {BigInt(1), BigInt(0)};
    auto prev = fib_fast(n - 1);
    BigInt Fn = prev.first + prev.second;
    return {Fn, prev.first};
}

```

```

int main() {
    for (int i = 0; i <= 25; i++) {
        int n = i * 20;
        auto result = fib_fast(n);
        cout << "F(" << n << ") = ";
        result.first.print();
        cout << endl;
    }
    return 0;
}

```

Time complexity = $O(n)$